From User Manuals to Mixed-Reality Simulations: A Conceptual Model and an AI-supported Authoring Tool

Anonymous Author(s)

Anonymous Institution

Abstract. Mixed-reality simulations of real-world devices can help train users to operate them, but developing such simulations requires expertise in 3D modelling, programming and game-engine development. This work explores how generative AI can make simulation development accessible to non-experts, starting from the device user manuals. It proposes an engine-independent conceptual model that captures the information needed to simulate a device, including its components, interactions and operating procedures. An AI-assisted authoring tool supports creating model instances from user manuals in three phases: extracting device information, modelling device behaviour, and generating a 3D representation. In each phase, generative AI produces an initial draft that users can review and refine. A feasibility study modelled three training simulations involving two devices that were selected in collaboration with organizations supporting people with cognitive impairments. All of them were successfully generated and executed on Meta Quest 3 using the model instances produced by the tool, demonstrating the technical feasibility of the approach.

Keywords: Mixed reality · Conceptual modelling · End-user development · Large language models · Authoring tools

1 Introduction

Mixed Reality (MR) simulations provide a safe, repeatable environment in which learners can build skills and confidence by rehearsing the operation of a device [9, 28, 29]. This is particularly relevant for people with intellectual disabilities and cognitive support needs, whose difficulties with working memory and task retention can make starting a new job challenging, limit their autonomy and increase the risk of social isolation [22]. They often require adapted training and individual support, which are not always available, and technical documentation is rarely a useful learning material [25].

Developing MR training simulations requires expertise in 3D modelling, behaviour programming, and game engines, skills that those who best understand the training needs, such as job coaches or occupational therapists, usually lack. Additionally, this effort recurs for every device in the training program. In this context, End-User Development (EUD) tools, which empower non-experts to create their own digital experiences [17], offer a promising way to democratise MR training simulation creation. As a first step towards these EUD tools, this research investigates how to integrate generative AI into the authoring process. Much of the information needed to simulate a device, such as its components, states and procedures, is already in its user manual. Rather than asking an

AI to generate the simulation directly, which yields code that a non-programmer cannot inspect, this work extracts that information into a conceptual model of the device, from which the simulation is generated. The author's role thus shifts from building the simulation to reviewing and refining AI-generated drafts. Our research was conducted in three stages: defining the information needed to simulate a device, developing an AI-assisted tool to generate model instances from user manuals, and building MR simulations from these instances.

This paper focuses on the first two stages: defining an engine-independent conceptual model of interactive devices and investigating how generative AI can support creating instances from user manuals. Its contributions are: **(i) a conceptual model** of interactive devices that defines the information required for simulation and its structure, together with competency questions to assess the model; **(ii) a three-phase method** for creating instances of this model from device user manuals, implemented in an AI-assisted authoring tool; and **(iii) a feasibility study** in which three simulations of two devices were created with the tool, assessing the validity of the resulting model instances and whether executable MR simulations can be generated from them. To select the devices to simulate, we consulted two associations: [anonymised1], which specialises in employment inclusion for people with borderline intellectual functioning (BIF), and [anonymised2], which supports people with disabilities in daily living activities. Together, we identified two devices relevant to training in different contexts: an office printer for workplace training and a domestic coffee machine for daily living skills.

The rest of the paper starts with the related work (Section 2), followed by the description of the conceptual model (Section 3), the AI-assisted authoring method and tool (Section 4), evaluation (Section 5), discussion and future work (Section 6), and conclusions (Section 7).

2 Background

2.1 End-User Development and Generative AI

EUD tools aim to empower people who are not professional programmers to create or adapt software [17]. Such users face recurrent barriers, from deciding what to express to understanding why a program fails [16]. EUD tools usually address these challenges by providing predefined components and behaviours, often through visual blocks or node graphs. In MR, this approach has enabled, for instance, end users to prototype cross-reality environments [3] and teachers to create augmented reality learning experiences [24]. While this reduces the expertise required, it also limits what users can create, provoking circumscribed thinking, premature fixation, and bounded ideation [23, 27]. Generative AI can overcome this limit by generating content and behaviours from natural-language descriptions [15], but its results are hard to verify and correct [18]. In this work, the LLM generates a structured, inspectable model instead of code, which the author reviews without requiring programming.

2.2 Authoring MR Experiences Without Programming Expertise

Authoring MR content remains difficult for non-programmers. DART [19], a toolkit that extended a multimedia authoring environment familiar to designers, enable them

to create AR experiences. However, a decade later, AR authoring remain largely inaccessible to non-technical users [13]. Other tools let authors create content within the immersive environment [20, 30] or constrain authoring to a genre, as TrainAR does for procedural AR training [4]. These tools lower the technical barrier to AR authoring, but authors must still specify objects and behaviours manually, and some require programming concepts (e.g., dataflow programming in FlowMatic) or game engines (e.g., Unity in TrainAR). All these tools do not use documentation, such as user manuals, to derive these specifications. A review of 69 AR authoring toolkits found that few reduced the need for programming skills [12]. Furthermore, these approaches require authors to provide procedural knowledge (i.e., which components to manipulate, in what order, and under which conditions). In contrast, our approach extracts this knowledge from manuals, shifting the author's role from modelling procedures to reviewing and validating AI-generated specifications.

2.3 3D Representations, Conceptual Models and LLMs

Several techniques exist to obtain 3D models of real objects, as reviewed in [8]. Photogrammetry yields faithful but monolithic meshes that must be split before their parts can move [6], while generative techniques such as NeRF and 3DGS lack geometric precision [2]. Since neither directly produces interactive objects with several components, this work treats geometry as a replaceable layer. Behaviour, in turn, can be described independently of its implementation through conceptual models such as VR-WISE [11] and finite state machines are a common formalism for procedural behaviour [26]. LLMs are used to extract structured information from documents [1] and to create virtual content [7, 10], but such content generation produces code or scene state that non-programmers can hardly inspect. In this work, the LLM drafts an instance of a conceptual model, which validators and the author check before a fixed runtime interprets it.

3 Conceptual Model of Interactive Devices

This section addresses the first objective of the paper: identifying the information required to specify a device simulation independently of its implementation. Because the simulations are intended for training, the model describes a device as a learner perceives and operates it, i.e. the parts that can be seen and manipulated, the states in which they can be observed, the actions that change those states and the procedures to be learnt.

3.1 Model scope

The model targets devices whose components have discrete, observable states that are changed by direct manipulation (pressing, turning, sliding or opening), whose tasks are finite sequences of steps with clear preconditions and whose operation is documented in illustrated manuals, such as printers or coffee machines. Process machinery, continuously controlled devices and safety-critical systems are out of scope. For clarity, we use the Nespresso Essenza Mini C30 coffee machine, one of the two devices identified with the associations, and its coffee preparation procedure as a running example.

3.2 The Model

The model is specified as a UML class diagram (Fig. 1) following the OMG UML 2.5.1 standard [21]. UML is independent of implementation technology and the class diagram provides the basis for the schemas used by the authoring tool to store model instances. In the diagram, filled diamonds denote composition, while slashes indicate derived associations.

Structure. A *Device*, with its `name` and physical `dimensions`, is composed of one or more *Components*, each with a *name* and a *location*. *Components* include both the device's parts and the objects used with it. A *Material* is a *Component* that the device processes rather than one of its parts, such as the capsule or the cup. The coffee machine has 27 *Components*, including its lever, LEDs and water tank.

Representation and Interaction. Each Component has exactly one *Representation* (a primitive `shape` with its `size`, `position` and `pivot`) which can later be replaced by a detailed mesh. A *Component* may have several *Interactions*, which define how the learner operates it in MR. Each *Interaction* has a `kind` (slide, hinged rotation, press or attachment to another component), a `range` and a `threshold` at which it *triggers* one or more *Transitions*. For example, the lever rotates up to 70° about its rear edge, and passing 85% of this range triggers `lift_lever`. *Interactions* thus connect the 3D representation with the behaviour.

Behaviour. A *Component* may have a *StateMachine*, which in turn consists of *StateVariables*, *States* and *Transitions*:

- A *StateVariable* has a `name`, its possible `values` and an `initialValue` (e.g. the lever's `position` is either closed or lifted).
- A *State* is a combination of variable values and may have a `timeout`, after which the machine moves to another *State* (e.g. after switching on, the LEDs blink for 25 s and then become steady).
- A *Transition* moves its machine from a *source* to a *target State* when its `action` is performed. It may have a `guard` over the variables of other components and may emit a `signal` (e.g. insert_capsule is guarded by machine_lever.position eq lifted).

An `action` whose `guard` is false leaves the state unchanged and is logged, so the guidance layer can give a hint. `Signals` inform the guidance and logging layers but never *trigger transitions*. In line with the training purpose, the model represents what the learner can observe: the machine's internal heating is not modelled but the LEDs that show it are.

Components without *StateMachines*, such as the coffee machine's internal brewing unit, are represented as static geometry but can be assigned behaviours if needed for future procedures.

Procedures. A *Device* is also composed of *Procedures*, which are the tasks to be learnt. A *Procedure* is composed of an ordered list of *Steps*, each with an `order`, an `instruction` from the manual and a `simplifiedInstruction` for learners who need more support. Each *Step* performs one or more alternative *Transitions*, unless it is `external` to the device. For example, coffee preparation has ten steps, from switching the machine on to ejecting the capsule. Steps 1 and 7 can each be completed with either coffee button. A *Procedure* has *preconditions* and *postconditions*, which are

StateConditions requiring a *StateVariable* to hold a `value` (e.g., the water tank must be filled). It is governed by *Rules* from the manual's warnings: a *Rule* with an `expression` is enforced through the `guards` of the *Transitions* it *appliesTo* (e.g., the drip tray must be in place before brewing), while one without an expression is documentary (e.g., never lift the lever during operation). A *Procedure* may require other *Procedures* to be completed (e.g. descaling requires the machine to be switched on). The derived association */involves* links a *Procedure* to the *Components* whose *Transitions* or *StateVariables* it uses.

3.3 Integrity Constraints

The UML class diagram (Fig. 1) defines the structure of model instances but does not guarantee their consistency or executability. For example, an instance may conform to the diagram but define a procedure in which inserting a capsule precedes lifting the lever, making the guard of `insert-capsule` impossible to satisfy. To prevent such inconsistencies, we defined five constraints that valid instances must satisfy:

- (C1) Every non-external *Step* performs at least one *Transition* defined in the instance.
- (C2) Every required *Procedure* exists, and its postconditions guarantee the preconditions of the requiring procedure.
- (C3) The initial values satisfy the preconditions of the first *Procedure*.
- (C4) Every *Procedure* is executable: each *Step* can perform at least one of its *Transitions* in sequence, and the *Procedure's* postconditions hold upon completion.
- (C5) Every *Step* has at least one *Transition* triggered by an *Interaction*.

These constraints detect missing or invalid transitions (C1), inconsistent procedure dependencies (C2), unreachable execution paths (C3–C4), and steps that cannot be performed through MR interactions (C5).

4 AI-Assisted Method for Creating Model Instances

We propose a three-phase method (Fig. 2) for creating instances of the conceptual model from device user manuals using generative AI. The method is implemented in an AI-assisted authoring tool that supports users throughout the process. This section presents the tool's workflow, details the three phases, describes the validation and integrity checks and explains how the resulting instances are used to build executable MR simulations.

4.1 Tool overview

The method is implemented as a web-based tool (Fig. 3). In each phase, the tool builds a few-shot prompt [5] combining information from the device manual and outputs from previous phases with predefined input-output examples and an output schema derived from the conceptual model. The author submits the prompt to a general-purpose LLM and imports its response into the tool. This approach requires no model-specific training and allows different LLMs to be used. The tool checks the structural validity of the response and displays the generated information in the corresponding phase's interface,

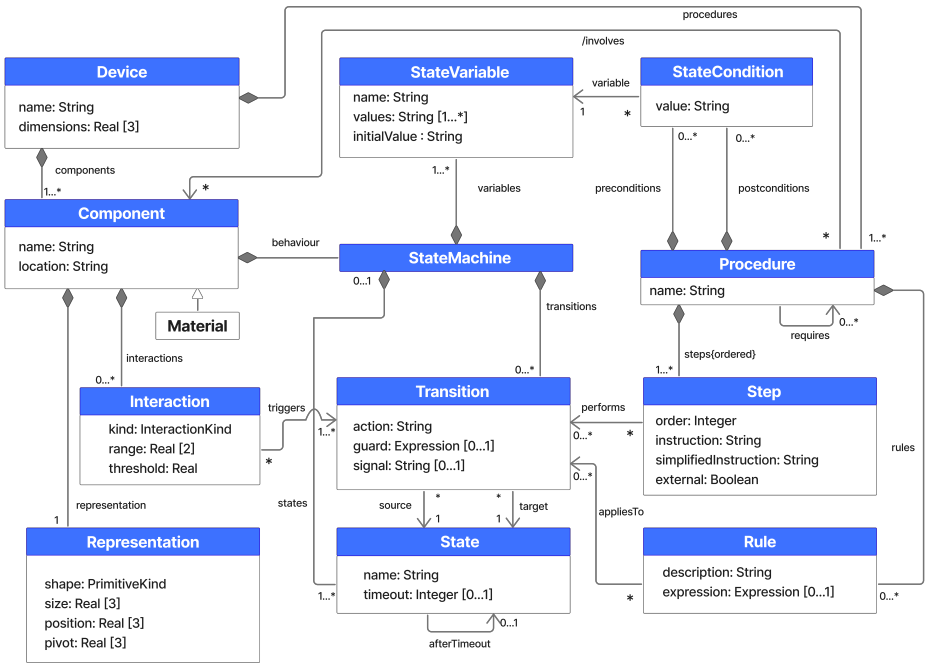


Fig. 1. Conceptual device model as a UML class diagram [21]. Composite ends have multiplicity 1. Branches and repetitions within procedures are omitted for clarity.

where the author can review, edit, add, merge, or remove elements. Corrections are applied to the model instance and recorded as instructions that can be incorporated into subsequent prompts, allowing the LLM to regenerate the output while preserving the author's decisions. Each value in the device specification records its source (manual, inference, library, author decision, or default), distinguishing information extracted from the manual from information obtained through other means. Library values come from a reference library of components, movements and layouts taken from previously validated instances (once an instance passes validation, its patterns can be exported to the library and reused for later devices of the same family), while default values are generic presets applied only when neither the manual nor the library provides one. The resulting data is stored in two JSON files: a behavioural specification containing components, state machines, and procedures; and a device specification containing representations and interactions.

4.2 Phase 1: Device Information Extraction

A sequence of few-shot LLM prompts extracts the device's main components, their characteristics and operating procedures from the user manual, organizing the information into two tables: components and procedures (Fig. 3B). Warnings and cautions in the manual are later incorporated as *Rules* associated with *Procedures* (Section 3).

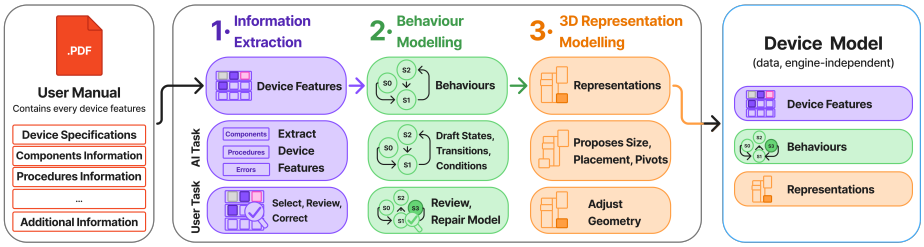


Fig. 2. Authoring method. Each phase instantiates a different part of the conceptual model (Fig. 1) using AI-generated drafts derived from the device's PDF user manual reviewed by the author.

The author reviews the extracted information, correcting component names, merging duplicates, and adding missing elements, then selects the procedures to simulate and identifies any prerequisites. This selection determines which components and interactions need to be modelled, while others can initially be represented as static geometry. This enables incremental modelling, focusing development on the selected procedures rather than the entire device. At the end of this phase, the model instance contains the *Device*, its *Components*, and the selected *Procedures*, whose *Steps* are still represented as textual descriptions.

4.3 Phase 2: Device Behaviour Modelling

The LLM drafts the *StateMachine* for each involved component, the *Transitions* performed by each *Step*, the procedure's conditions, *Rules*, dependencies, and a *simplifiedInstruction* for each step (e.g., "Pull the lever on top all the way up"). For example, "press either the Espresso or the Lungo button; the lights blink while heating up (25s)" is translated into a timed LED state and a *Step* with two alternative *Transitions*. The author reviews and edits the procedure as a flow diagram (Fig. 3C), removing, reordering or adding steps. The edited flow, together with the component names established in Phase 1, is then provided to the LLM to regenerate the specification while preserving the author's changes. The integrity checks are subsequently run again on the updated specification (Section 4.5).

4.4 Phase 3: 3D Representation Generation

The tool generates an initial 3D *Representation* of each component using geometric primitives (blocks) to approximate the device's shape, dimensions and spatial arrangement. Component dimensions are derived from information extracted in Phase 1, when available, while their relative positions are estimated from illustrations in the device manual. The author reviews and refines the resulting model in a 3D editor (Fig. 3D), where blocks can be moved, resized, rotated, hidden or added to complete and correct the representation.

For interactive components, the tool suggests *Interactions* from a set of predefined, reusable types, including sliding, hinged rotation, pressing and attaching objects to

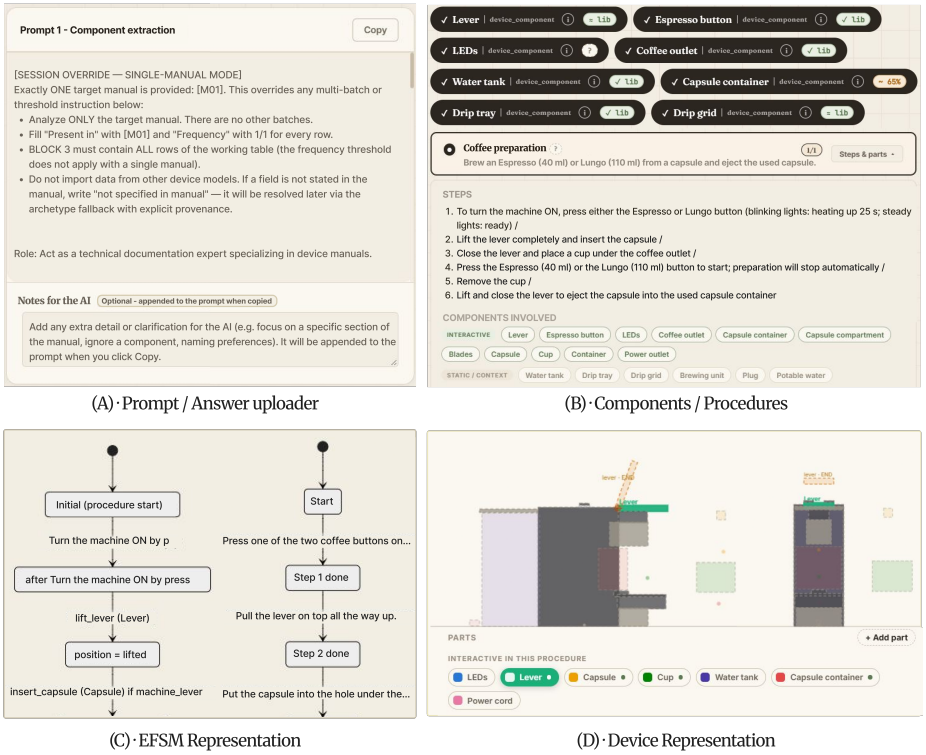


Fig. 3. The authoring tool. (A) Prompt composed by the tool with optional notes for the AI and upload of the model's response. (B) Extracted components tagged with their provenance and procedures with the components each one involves. (C) A procedure as a flow of situations and actions, editable by the author. (D) Definition of the 3D representation, where the author selects a part and sets its start and end poses.

other components. The author can refine these interactions by specifying the initial and final poses of each movement. Each *Interaction* is linked to *Transitions* in the behavioural specification, allowing user actions on the 3D representation to trigger the corresponding state changes. For example, lifting the coffee machine's lever beyond 85% of its maximum rotation angle (70°) triggers the `lift-lever` transition. At the end of this phase, the model instance includes the components' 3D *Representations* and their associated *Interactions*.

4.5 Validation and Integrity Checking

The authoring tool supports the review of AI-generated instances through automatic checks and safeguards during editing. Warnings identify potential problems, such as duplicate components, missing descriptions or a high proportion of inferred values, without blocking the authoring process. Errors indicate violations of the integrity constraints defined in Section 3.2.

After generating or modifying the behavioural specification, the tool checks constraints C1–C4 by resolving references and replaying procedures from their initial states, evaluating guards and timeouts. Violations are reported with the affected steps and transitions. Before export, the tool checks C5 and verifies that each *Interaction* triggers an existing *Transition*. The tool also requires confirmation of potentially disruptive edits and prevents components needed by the selected procedures from being removed from the simulation. Export is blocked until all integrity constraints are satisfied and the author confirms the representation against the device manual.

4.6 From Model Instances to MR Simulations

Once a model instance has been created, it can be used to build an executable MR simulation. To support this process, we developed a semi-automatic Unity-based builder that combines a device-independent runtime with manual configuration steps. The author imports the JSON files produced by the authoring tool into Unity, where the builder creates and positions the components' 3D representations and links each *Interaction* to the *Transitions* it triggers. The author then configures hand interactions using Unity's XR Interaction Toolkit, following a checklist generated by the tool for each interactive component.

The runtime also provides a guidance layer that highlights the component to operate and displays the corresponding instruction at each procedure step. After repeated unsuccessful attempts, it displays the *simplifiedInstruction*, providing additional guidance without time limits or penalties.

The primitive geometry can be replaced with externally generated 3D meshes without modifying the behavioural specification (Fig. 4). Configuring hand interactions and replacing primitive geometry currently require manual intervention, and automating these steps remains future work.

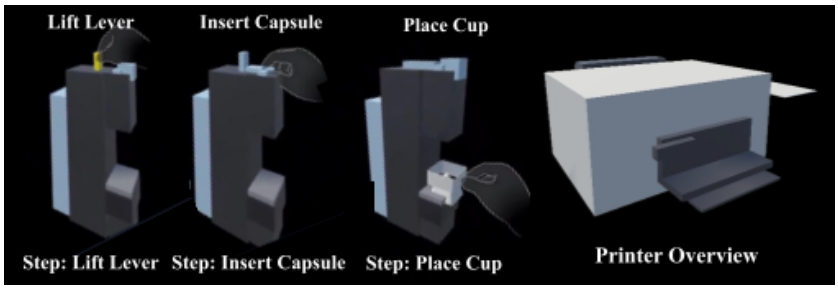


Fig. 4. Simulations on Meta Quest 3. Left: hand interactions during coffee preparation; the cup is an externally generated mesh (a lamp). Right: the printer instance model.

5 Feasibility Study

The feasibility study examines the model's expressiveness, the consistency of tool-generated instances and the feasibility of building executable MR simulations from them. Tool usability is outside its scope. Three instances were created for two devices identified in collaboration with the participating associations: a Nespresso Essenza Mini C30 coffee machine (home), covering coffee preparation and descaling, and an HP DeskJet 2720e printer (work), covering paper loading. Descaling was selected to test the representation of procedural prerequisites and dependencies.

One author created the instances using the authoring tool and Claude Opus 5.5. The simulations were built with Unity 2022.3 LTS and Meta XR SDK 201.0.0 and ran on Meta Quest 3. Prompts and files are available in the replication package[1].

The evaluation comprised three levels:

Model expressiveness. Following the competency-question approach from ontology engineering [14], fourteen questions assessed whether the model captures the information needed to simulate devices within its scope (Table 1), covering all classes and associations in the conceptual model in Fig. 1. An automated validator queried each instance to answer them. For example, CQ4 identifies transitions available from a state and their guards: for a capsule in the `outside` state, it returns `insert-capsule`, guarded by the lever being lifted. Questions involving relationships absent from an instance were marked `n/a`.

Instance consistency. Integrity constraints C1–C5 checked the generated instances for inconsistencies not detected by competency questions, such as non-executable procedures.

Technical feasibility. Each instance was used to build an MR simulation on Meta Quest 3. Two authors with Unity experience executed the procedures using hand interactions and verified the resulting state changes against the device manuals (Fig. 4).

5.1 Results

Table 2 summarises the three instances and their validation results. Their complexity ranged from 6 steps and 12 transitions (paper loading) to 22 steps and 24 transitions (descaling). All applicable competency questions were answered (13/14 for single-procedure instances and 14/14 for descaling), without requiring additional model elements. All instances included materials and cross-guarded transitions, while both coffee-machine instances also included timed states.

The initial descaling instance failed the integrity check because of a missing prerequisite procedure and omitted intermediate actions in repeated steps, neither detected by the competency questions. After revising the behaviour prompt, the regenerated instance passed all checks, as did the other two. All three simulations ran on Meta Quest 3, where two researchers successfully completed the procedures using hand interactions.

Across the three authoring phases, the LLM drafts required corrections, mainly because of implicit or ambiguous information in the manuals. During extraction, the author

[1] https://anonymous.4open.science/r/MedCHI26_replicationpackage-E7BA/

Table 1. Competency questions used to evaluate the model, with example answers for the coffee machine.

ID	Competency question	E.g. in Coffee Machine
CQ1	Which *Components* does a *Device* comprise and which are *Materials* or have a *StateMachine*?	27 components; 8 with behaviour (2 materials)
CQ2	What is the *Representation* (shape, size, position, pivot) of a *Component*?	Lever: $50 \times 12 \times 99$ mm, rear-hinge pivot
CQ3	Which *StateVariables* and *States* does a *StateMachine* define, and what are their initial values?	Lever: closed/lifted (initial closed)
CQ4	Which *Transitions* leave a given *State* and under which guard?	insert_capsule if lever *lifted*
CQ5	Which *Interaction* triggers a *Transition*?	lift_lever: lever hinged rotation
CQ6	Which *Transitions* are guarded by other *Components* and which signals do they emit?	4 of 17; insert_capsule emits capsule_inserted
CQ7	Which *Procedures* does a *Device* comprise?	Coffee preparation
CQ8	Which *Steps* form a *Procedure*, in which order, and which *Transitions* do they perform?	10 steps; step 4: insert_capsule
CQ9	Which *StateConditions* are the pre- and postconditions of a *Procedure*?	Pre: tank filled, ...; Post: cup removed, ...
CQ10	Which *Components* does a *Procedure* involve?	6 components (4 acted on, 2 in conditions)
CQ11	Which *Rules* govern a *Procedure* and which can be evaluated?	11 rules, 6 evaluable
CQ12	Which *Procedures* does a *Procedure* require?	n/a (single procedure)
CQ13	Which state variables, actions and signals does a *Device* use?	4 variables, 12 actions, 16 signals
CQ14	Which *Materials* does a *Device* process and which values can their variables take?	Cup: position×content; Capsule: outside, in_compartment, ejected

distinguished user-operated components from internal mechanisms and packaging contents. During behaviour modelling, the LLM missed some procedural dependencies and treated one optional step as mandatory. During 3D modelling, component positions were estimated from illustrations, but depth could not be reliably inferred from 2D images.

Table 2. Characteristics of the three model instances and results of their validation. Parentheses indicate subsets of the corresponding totals.

	Coffee machine		Printer
Instance			
Procedure	Prepare coffee	Descale	Load paper
Procedures (dependent)	1 (0)	2 (1)	1 (0)
Components with behaviour (materials)	8 (2)	11 (2)	5 (1)
Static components	19	15	41
Transitions (cross-guarded)	17 (4)	24 (8)	12 (8)
Steps (alternative / repeated)	10 (2/0)	22 (1/1)	6 (0/0)
Rules (evaluable)	11 (6)	11 (4)	5 (1)
Verification results			
Competency questions (met / n.a.)	13 / 1	14 / 0	13 / 1

6 Discussion and Future Work

The feasibility study provides initial evidence that the model can represent devices from work and home settings without requiring modifications, and that LLM-assisted authoring can produce instances from manuals that can be used to build executable MR simulations. However, model conformity does not guarantee instance consistency, highlighting the need for integrity checks and human review. The results also suggest

where human oversight is most needed in AI-assisted authoring: the LLM handled explicitly documented information, such as components, action sequences and timings, more reliably than implicit information, such as procedural dependencies, optional steps and spatial relationships. Authoring tools should therefore focus validation and human review on these less explicit aspects.

The study was intended to assess the feasibility of the proposed approach rather than provide an exhaustive evaluation of the model and tool. The instances were created by researchers familiar with both, so usability by non-programmers remains to be evaluated. Although the devices exercised several modelling capabilities, others, such as conditional branching and delegation between procedures, were not tested, and procedural dependencies were examined in only one device. The competency questions were designed to cover every class and association of the model, so answering them systematically exercises all its elements in each instance; complementing them with requirements defined by domain experts, such as the associations, would further strengthen this evidence. The model currently focuses on discrete states and fixed delays rather than continuous processes or concurrency.

Finally, the tool generates approximate 3D representations using geometric primitives, which are sufficient for verifying simulation structure and interactions but may lack the realism needed for training. However, these can be replaced with more realistic models [8] without changing the underlying behaviour.

7 Conclusion

This paper investigated how generative AI can support the creation of MR simulations of real devices from user manuals. It proposed an engine-independent conceptual model and an AI-assisted tool for generating its instances from manuals. A feasibility study involving an office printer and a domestic coffee machine, selected in collaboration with associations supporting people with cognitive impairments, provided initial evidence that the model could represent both devices and that the resulting instances could be used to generate executable MR simulations on Meta Quest 3. Although the LLM generated the initial drafts, human review and integrity checks were needed to identify and correct errors. By shifting the effort from programming to reviewing AI-generated specifications, the approach could let educators create MR training for the devices their learners use at a lower cost, making it affordable for organizations with limited resources, ultimately extending its reach to more learners. For workers with cognitive impairments, this could support integration and solidarity, two key values in the Mediterranean area.

Future work will evaluate the authoring tool with non-programmers and the resulting simulations with learners with intellectual disabilities, including the effectiveness of the guidance provided during training. Technical developments will focus on automating the remaining Unity integration steps and extending the behavioural model to support event-based interactions between components.

References

1. Ateia, S., Kruschwitz, U., Scholz, M., Koschmider, A., Almohaishi, M.: LLM-based information extraction to support scientific literature research and publication workflows. In: New

Trends in Theory and Practice of Digital Libraries. pp. 90–99. Springer Nature Switzerland, Cham (2026)
2. Bai, S., Li, J.: Progress and prospects in 3D generative AI: A technical overview including 3D human (2024), https://arxiv.org/abs/2401.02620
3. Bellucci, A., Zarraonandia, T., Díaz, P., Aedo, I.: End-user prototyping of cross-reality environments. In: Proceedings of the Eleventh International Conference on Tangible, Embedded, and Embodied Interaction. pp. 173–182. TEI '17, ACM (2017). https://doi.org/10.1145/3024969.3024975
4. Blattgerste, J., Behrends, J., Pfeiffer, T.: TrainAR: An open-source visual scripting-based authoring tool for procedural mobile augmented reality trainings. Information **14**(4), 219 (2023). https://doi.org/10.3390/info14040219
5. Brown, T.B., et al.: Language models are few-shot learners. CoRR **abs/2005.14165** (2020), https://arxiv.org/abs/2005.14165
6. Buonamici, F., Carfagni, M., Furferi, R., Governi, L., Lapini, A., Volpe, Y.: Reverse engineering modeling methods and tools: a survey. Computer-Aided Design and Applications **15**(3), 443–464 (2018). https://doi.org/10.1080/16864360.2017.1397894
7. Cuervo-Rosillo, R., Zarraonandia, T., Díaz, P.: Using generative AI to support non-experts in the creation of immersive experiences. In: Proceedings of the 2024 International Conference on Advanced Visual Interfaces. AVI '24, ACM (2024). https://doi.org/10.1145/3656650.3656733
8. Cuervo-Rosillo, R., Zarraonandía, T., Díaz, P.: Understanding the potential of artificial intelligence to semi-automatically generate realistic 3D interactive models of composite objects. In: Proceedings of the 2026 International Conference on Advanced Visual Interfaces. AVI '26, ACM (2026). https://doi.org/10.1145/3811427.3811463
9. Dalgarno, B., Lee, M.J.W.: What are the learning affordances of 3-D virtual environments? British Journal of Educational Technology **41**(1), 10–32 (2010). https://doi.org/10.1111/j.1467-8535.2009.01038.x
10. De La Torre, F., Fang, C.M., Huang, H., Banburski-Fahey, A., Amores Fernandez, J., Lanier, J.: LLMR: Real-time prompting of interactive worlds using large language models. In: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24). ACM (2024). https://doi.org/10.1145/3613904.3642579
11. De Troyer, O., Kleinermann, F., Pellens, B., Bille, W.: Conceptual modeling for virtual reality. In: Tutorials, Posters, Panels and Industrial Contributions at the 26th International Conference on Conceptual Modeling - Volume 83. p. 3–18. ER '07, Australian Computer Society, Inc., AUS (2007)
12. Dengel, A., Iqbal, M.Z., Grafe, S., Mangina, E.: A review on augmented reality authoring toolkits for education. Frontiers in Virtual Reality **3** (2022). https://doi.org/10.3389/frvir.2022.798032
13. Gandy, M., MacIntyre, B.: Designer's augmented reality toolkit, ten years later: implications for new media authoring tools. In: Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (UIST '14). ACM (2014). https://doi.org/10.1145/2642918.2647369
14. Järvenpää, E., Siltala, N., Hylli, O., Lanz, M.: The development of an ontology for describing the capabilities of manufacturing resources. Journal of Intelligent Manufacturing **30**(2), 959–978 (2019). https://doi.org/10.1007/s10845-018-1427-6
15. Jiang, E., Toh, E., Molina, A., Olson, K., Kayacik, C., Donsbach, A., Cai, C.J., Terry, M.: Discovering the syntax and strategies of natural language programming with generative language models. In: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. CHI '22, ACM (2022). https://doi.org/10.1145/3491102.3501870

16. Ko, A.J., Myers, B.A., Aung, H.H.: Six learning barriers in end-user programming systems. In: 2004 IEEE Symposium on Visual Languages - Human Centric Computing (VL/HCC). pp. 199–206. IEEE (2004). https://doi.org/10.1109/VLHCC.2004.47
17. Lieberman, H., Paternò, F., Klann, M., Wulf, V.: End-user development: An emerging paradigm. In: End User Development, Human-Computer Interaction Series, vol. 9, pp. 1–8. Springer, Dordrecht (2006). https://doi.org/10.1007/1-4020-5386-X_1
18. Liu, M.X., Sarkar, A., Negreanu, C., Zorn, B., Williams, J., Toronto, N., Gordon, A.D.: "What It Wants Me To Say": Bridging the abstraction gap between end-user programmers and code-generating large language models. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23). ACM (2023). https://doi.org/10.1145/3544548.3580817
19. MacIntyre, B., Gandy, M., Dow, S., Bolter, J.D.: DART: A toolkit for rapid design exploration of augmented reality experiences. In: Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST '04). pp. 197–206. ACM (2004). https://doi.org/10.1145/1029632.1029669
20. Ng, G., Shin, J.G., Plopski, A., Sandor, C., Saakes, D.: Situated game level editing in augmented reality. In: Proceedings of the Twelfth International Conference on Tangible, Embedded, and Embodied Interaction. p. 409–418. TEI '18, ACM (2018). https://doi.org/10.1145/3173225.3173230
21. Object Management Group: OMG Unified Modeling Language (OMG UML), Version 2.5.1. Object Management Group (Dec 2017), https://www.omg.org/spec/UML/2.5.1
22. Peltopuro, M., Vesala, H.T., Ahonen, T., Närhi, V.M.: Borderline intellectual functioning: an increased risk of severe psychiatric problems and inability to work. Journal of Intellectual Disability Research **64**(12), 923–933 (2020). https://doi.org/10.1111/jir.12783
23. Robertson, B., Radcliffe, D.: Impact of cad tools on creative problem solving in engineering design. Computer-aided design **41**(3), 136–146 (2009)
24. Romano, M., Díaz, P., Aedo, I.: Empowering teachers to create augmented reality experiences: the effects on the educational experience. Interactive Learning Environments **31**(3), 1546–1563 (2023). https://doi.org/10.1080/10494820.2020.1851727
25. Sutherland, R.J., Isherwood, T.: The evidence for easy-read for people with intellectual disabilities: A systematic literature review. Journal of Policy and Practice in Intellectual Disabilities **13**(4), 297–310 (2016). https://doi.org/10.1111/jppi.12201
26. Utting, M., Legeard, B.: A model of your system. In: Practical Model-Based Testing, pp. 59–105. Morgan Kaufmann, San Francisco (2007). https://doi.org/10.1016/B978-012372501-1/50004-1
27. Weintrop, D., Wilensky, U.: Comparing block-based and text-based programming in high school computer science classrooms. ACM Transactions on Computing Education **18**(1) (2017). https://doi.org/10.1145/3089799
28. Xie, B., Liu, H., Alghofaili, R., Zhang, Y., Jiang, Y., Lobo, F.D., Li, C., Li, W., Huang, H., Akdere, M., Mousas, C., Yu, L.F.: A review on virtual reality skill training applications. Frontiers in Virtual Reality **2**, 645153 (2021). https://doi.org/10.3389/frvir.2021.645153
29. Zarraonandia, T., Cuervo-Rosillo, R., Montero, A., Díaz, P.: Intelligent mixed reality twins of workplace equipment for training individuals with borderline intellectual functioning. In: Proceedings of the 16th Biannual Conference of the Italian SIGCHI Chapter. ACM (2025). https://doi.org/10.1145/3750069.3750300
30. Zhang, L., Oney, S.: FlowMatic: An immersive authoring tool for creating interactive scenes in virtual reality. In: Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20). pp. 342–353. ACM (2020). https://doi.org/10.1145/3379337.3415824